Contrast Ratio Checker — Requirements Document

Product: Free Tools suite (alongside VPAT Generator, ACR Generator) **Tool name (working):** Contrast Ratio Checker **Owner:** [you] **Status:** Draft v1 — for team review **Reference benchmark:** WebAIM Contrast Checker (feature parity + extensions)

---

1. Overview & Goals

A free, client-side web tool that lets users check the color contrast ratio between a foreground and background color, see pass/fail results against WCAG, and fix failing combinations. It extends the familiar WebAIM experience with palette suggestions and automatic "nudge to nearest passing color."

Primary goals - Match WebAIM's core experience so it feels instantly familiar. - Go beyond WebAIM with the four extensions above. - Serve as a lead magnet for the accessibility product — fast, accurate, no signup. - Be itself fully accessible (we are an accessibility company; this tool is a credibility statement).

Success metrics (suggested) - Result recalculates in < 50 ms on color change (feels instant). - Tool passes its own WCAG 2.2 AA audit with zero issues. - Measurable click-through from the tool to the main product CTA.

---

2. Scope

In scope

- Core foreground/background contrast checker with live results.
- Multiple color input methods (see FR-2).
- Results for normal text, large text, and graphical objects / UI components — at both AA and AAA where applicable.
- Lightness sliders + "nudge to nearest passing color."
- Accessible palette suggestions.
- WCAG 2.1 **and** 2.2 (same contrast engine — see §3).

Out of scope (v1)

- **Shareable permalink** — removed from v1.
- **Eyedropper (EyeDropper API)** — removed from v1.
- **UI-component / `1.4.11` preview illustration** — removed from v1 (the `1.4.11` 3:1 result row in §3.3 is retained; only the visual preview widget is dropped).
- **Bulk / multiple-pair checking and CSV import/export** — removed from v1. Single-pair checking only. Candidate for a later phase.
- **Image upload & background color sampling** — removed from v1. Candidate for a later phase.

- **APCA / WCAG 3 contrast algorithm** — not requested. Flag as a candidate for v2; the UI should leave room for a future "algorithm" toggle.
- User accounts, saved history server-side, or any login.
- Full-page/site scanning (that's the main product's job, not this free tool).

---

3. Standards & Calculation Engine

3.1 One engine for 2.1 and 2.2

WCAG 2.1 and 2.2 share the identical contrast formula and thresholds. Version 2.2 introduced new success criteria but did **not** change `1.4.3 Contrast (Minimum)`, `1.4.6 Contrast (Enhanced)`, or `1.4.11 Non-text Contrast`. Implement a single ratio engine; the "2.1 vs 2.2" distinction is purely a labeling/citation concern in the UI and report output.

3.2 Ratio formula (must match WCAG exactly)

1. Normalize each sRGB channel to the 0–1 range.
2. Linearize each channel: `c_lin = c/12.92 if c ≤ 0.03928 else ((c + 0.055)/1.055) ^ 2.4`
3. Relative luminance: `L = 0.2126·R_lin + 0.7152·G_lin + 0.0722·B_lin`
4. Contrast ratio: `(L_lighter + 0.05) / (L_darker + 0.05)`
5. Display rounded to two decimals (e.g. `4.53:1`), but compute pass/fail on the **unrounded** value to avoid edge-case mismatches with WebAIM.

3.3 Pass thresholds

Content type	AA	AAA
Normal text	4.5:1	7:1
Large text (≥ 18pt / 24px, or ≥ 14pt / 18.66px bold)	3:1	4.5:1
Graphical objects & UI components (`1.4.11`)	3:1	— (no AAA defined)

Each result shows a clear **Pass / Fail** state with text label + icon (never color alone).

---

4. Functional Requirements

FR-1 — Core checker

- FR-1.1 Two color fields: **Foreground (text)** and **Background**.
- FR-1.2 Live recalculation on every change (debounced ~50 ms).
- FR-1.3 Large numeric display of the contrast ratio (e.g. `7.21:1`).
- FR-1.4 Pass/Fail badges for each row in the §3.3 table.
- FR-1.5 A live **preview pane** rendering sample normal text and large text using the chosen colors.

FR-2 — Color input (recommended UX — "best flow")

Support all of the following, kept in sync (changing one updates the others): - FR-2.1 **Hex** input (3-, 6-, 8-digit; 8-digit alpha handled per FR-2.6). - FR-2.2 **RGB** and **HSL** inputs. - FR-2.3 **CSS named colors** (autocomplete from the 148 named colors). - FR-2.4 **Native color picker** swatch (`<input type="color">`). - FR-2.5 **Lightness sliders** (HSL lightness) under each color — drag to lighten/darken while watching the ratio update live. This is one of WebAIM's most-used features; replicate it. - FR-2.6 **Alpha / transparency:** decide policy — recommended v1: treat colors as opaque; if alpha < 1 is entered, composite over the other color (or over white) and show a note. (Open question — see §9.) - FR-2.7 Validation: invalid input shows an inline, accessible error and does not break the calculation.

FR-3 — Results display

- FR-3.1 Show all of: Normal text (AA, AAA), Large text (AA, AAA), Graphical/UI components (AA).
- FR-3.2 Live recalculation (per FR-1.2).
- FR-3.3 Swap foreground/background button (one click).
- FR-3.4 Copy-to-clipboard for each color value and for the full result summary.

FR-4 — Lightness sliders & "nudge to nearest passing color"

- FR-4.1 Sliders as in FR-2.5.
- FR-4.2 A **"Make it pass"** action per target level (e.g. "Fix for AA normal text"). It adjusts the chosen color by the **minimum visible change** needed to reach the target ratio.
- FR-4.3 The user chooses **which color to adjust** (foreground or background) before nudging.
- FR-4.4 Show before/after swatches and the new hex so the change is transparent and reversible (undo).

Algorithm (FR-4.2 / FR-5): 1. Convert the color to be adjusted into a lightness-separable space — **OKLCH preferred** (perceptually even), HSL acceptable. Hold **hue and chroma/saturation fixed**; vary **lightness only** so the result stays on the same hue family. 2. Contrast ratio is **monotonic** in the adjusted color's lightness (as it moves away from the fixed color's luminance, the ratio only increases). Use a **binary search** over lightness to find the smallest shift that reaches the exact target ratio (4.5 / 3 / 7). This is the "nearest passing color." 3. Compute pass/fail on the unrounded ratio (per §3.2). 4. **Edge case:** if the target is unreachable by moving one color alone (it would require going past pure black or white), clamp at the limit and either (a) fall back to adjusting both colors, or (b) report that the target is only achievable at black/white. Never return a non-passing "suggestion."

FR-5 — Palette suggestions

- FR-5.1 Given a failing pair, suggest a small set (e.g. 4–8) of accessible alternatives that pass the selected target, each computed via the FR-4 algorithm (so every suggestion is guaranteed to pass, not guessed).
- FR-5.2 Suggestions should stay close to the original colors (preserve hue family) so they remain on-brand. Offer at least three groupings: "keep foreground, fix background," "keep background, fix foreground," and "adjust both."
- FR-5.3 Each suggestion is one-click applyable and shows its resulting ratio.

FR-6 — Preview

- FR-6.1 Live sample of normal and large text in the chosen colors.

---

5. Recommended UX Flow

1. Land on the tool → two color fields pre-filled with a sensible default pair (one passing, to demonstrate). Result and preview visible immediately above the fold.
2. User edits a color via any input method → ratio, badges, and preview update live.
3. If a combination fails, contextual help appears: lightness sliders, "Make it pass," and palette suggestions — without scrolling away from the result.

Keep the tool as simple as WebAIM for the common single-pair case.

---

6. Non-Functional Requirements

6.1 Accessibility of the tool itself (high priority)

- Target: **WCAG 2.2 Level AA**, ideally AAA where reasonable.
- Full keyboard operability (sliders, inputs, controls).
- Results announced to screen readers via an `aria-live` region when colors change.
- No information conveyed by color alone — always pair with text + icon.
- Visible focus indicators meeting `2.4.11/2.4.13`.
- Sliders use proper `role` / `aria-valuenow` /labels.

6.2 Performance

- Recalculation < 50 ms; no layout jank while dragging sliders.

6.3 Privacy

- 100% client-side computation. No color data sent to any server. State this in the UI as a trust signal.

6.4 Browser & device support

- Latest 2 versions of Chrome, Edge, Firefox, Safari; responsive mobile/tablet.

6.5 Internationalization (optional v1)

- Strings externalized to allow future translation. English-only at launch unless decided otherwise.

---

7. Technical Architecture

- **Framework:** Next.js (matches existing free tools). Recommend the tool be a route within the existing app, e.g. `/tools/contrast-checker`.

- **Rendering:** primarily client-side for the interactive calculator; page shell can be statically generated for SEO.
- **No backend required.** All math runs in the browser.
- **Suggested libraries:** a small color manipulation library (e.g. `culori` or `colord`) for parsing/conversion and HSL math; avoid heavy dependencies. The WCAG luminance/ratio math is simple enough to implement directly (per §3.2) to guarantee exact correctness.
- **Testing:** unit tests for the ratio engine against known WebAIM values (lock in parity), plus accessibility tests (axe) in CI.

---

8. Placement & Integration

- Lives in the same Free Tools area as the VPAT Generator and ACR Generator.
- Shares the existing layout, header, navigation, and footer.
- Cross-link from/to the other free tools.
- Apply existing brand design system (colors, type, components).
- Include a product CTA consistent with the other tools (e.g. "Audit your whole site with [product]").

---

9. Assumptions & Open Questions

These were not explicitly specified — defaults assumed, please confirm or override:

1. **APCA out of scope** for v1 (WCAG 2.x only). Confirm.
2. **Alpha/transparency handling** (FR-2.7): default = composite semi-transparent colors before checking, with a note. Confirm desired behavior.
3. **Branding & SEO content:** assumed yes to brand design system, and yes to including educational SEO content on the page (what contrast is, the WCAG criteria, large-text definition) plus a product CTA. Confirm.
4. **Tool's own accessibility target:** assumed WCAG 2.2 AA (with AAA where feasible). Confirm.
5. **Browser support / mobile:** assumed last 2 versions of major browsers + responsive. Confirm any specific targets.
6. **i18n:** assumed English-only at launch, strings externalized for later. Confirm.
7. **Default starting colors** on load: pick a brand-appropriate passing pair.

---

10. Acceptance Criteria (sample)

- Ratio output matches WebAIM to two decimals across a test set of known pairs.
- All §3.3 rows show correct Pass/Fail at AA and AAA.
- Every input method (hex/RGB/HSL/named/picker/slider) updates all others and the result live.
- "Make it pass" produces a passing combination with the minimal visible change, and is undoable.
- Palette suggestions all pass the selected target and can be applied in one click.

- The tool passes an automated + manual WCAG 2.2 AA audit with zero blocking issues.
- All computation happens client-side (verified: no network calls with color data).